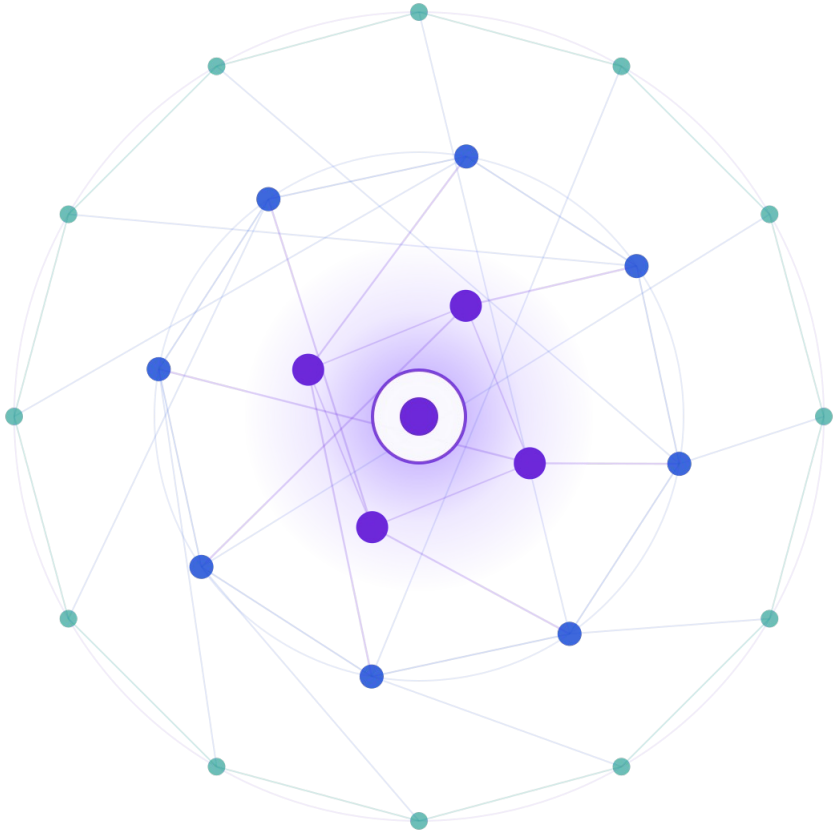
SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Theme 01
- Team Name - Doctors
- College Name - VIT Vellore
- Manish H. · manish.h2024@vitstudent.ac.in
- Aryan Sinha · aryan.sinha2024@vitstudent.ac.in
- Munal Singh · munal.singh2024@vitstudent.ac.in
- Anshul Singh · anshul.singh2024@vitstudent.ac.in



Submission GitHub: CodeDoctor repository

Theme

- **Theme 01: Agentic Code Intelligence** Natural-language code retrieval across complex codebases requires semantic understanding and exact structural evidence.
- **Judge Mandate: Pre/Post-Processing Focus** Evaluated on pre/post-processing innovation with strict CPU-only retrieval (minimal GPU, no external query APIs).
- **Core Objective** Return ranked, checkable code spans with file/line evidence rather than opaque, unverified generated answers.
- **Three-Tier Evaluation Goals** P0: CoIR AppsRetrieval benchmark (3,765 test queries / 8,765 docs); P1: Sub-second CPU incremental re-indexing; Bonus: Cross-version variant selection.

Existing Solutions & Gaps

- **Lexical vs. Semantic Tradeoff** Keywords (BM25) miss conceptual intent; dense models capture semantics but miss exact identifiers and incur high CPU latency.
- **Dense CPU Latency Bottleneck** Encoding every query on CPU takes 2-8 seconds; invoking large models or cloud APIs on every query violates hackathon efficiency goals.
- **Re-Indexing Redundancy in CI/CD** Standard code search engines re-embed full files or the whole repository on every commit, wasting compute on unchanged code.
- **Context Leakage & Unverified Code** Cross-commit search risks silent fallback to HEAD; text embeddings rank buggy near-misses above working fixes in 67-78% of queries.

Our Solutions & Architecture Diagram

Our Pre/Post-Processing Architecture

1. PREPROCESS (Offline)

- Exact CoIR / repo normalization
- Tree-sitter AST statement chunking*
- Content-addressed vector cache
- Invariant recipe keys (SHA-256)
- Incremental diff vector reuse*

2. RETRIEVE (CPU-Only)

- Evidence-gated router (0.4 ms free passes)*
- Pinned open-weights F2LLM-v2-1.7B
- Top-20 cosine candidate generation
- Additive Reciprocal Rank Fusion (k=60)*
- Zero-vector attention cap on >6k chars

3. POSTPROCESS (Dynamic)

- Multi-layout worked-example parser
- Defensive subprocess sandbox (2s timeout)
- Behavioral reranking (1e-06 tolerance)
- +0.02802 NDCG@10 ablation lift
- Lineage variant selection (100% vs 8.3%)*

* Active in local repository workbench & multi-version intelligence. P0 benchmark executes corpus vectors + behavioral execution reranking. Complies with Samsung CPU-only retrieval & permissible open-weights model guidelines.

Demo & Product Walkthrough

- **1. Open & Auto-Index** Open any repository in CodeDoctor workbench; Tree-sitter parses AST chunks and builds the content-addressed index in the background.
- **2. Multi-Signal Query** Router runs cheap lexical passes first; symbol queries resolve in 0.4 ms, while prose queries escalate to dense semantic retrieval.
- **3. Edit & Rebuild (P1)** Content-addressed cache re-embeds only modified functions, reusing unchanged vectors in 1.51 s (~68x CPU speedup).
- **4. Version Verification (Bonus)** Execution sandbox runs candidate variants across git history against worked examples, selecting the working variant with 100% accuracy.
- **Measured Latencies & Demo** Symbol lookup ≈ 0.4 ms · Cold build ≈ 5.6 s · Incremental update ≈ 1.51 s (Kaggle Xeon). Demo video: [5-Min Walkthrough Link].

Tools and tech stack used

- **Language & Core Engine** Python 3.11+, C++17 core engine (libcodedoctor) for native symbol indexing and graph traversal.
- **AST & Static Analysis** Tree-sitter (python, javascript, typescript); statement-boundary splitter for functions >80 lines; structural edge extractor.
- **Retrieval & Fusion** BM25, exact symbol table, dense cosine similarity, Reciprocal Rank Fusion (k=60) with additive component transparency.
- **Encoders & Vector Cache** codefuse-ai/F2LLM-v2-1.7B (accuracy tier), 0.6B (interactive tier); SQLite content-addressed EmbeddingCache (SHA-256).
- **Evaluation & Testing** Official MTEB 2.21.0 (cache=None), FastAPI + Uvicorn, 571 automated pytest tests, multi-platform GitHub Actions CI.

Impact & Use case

- **Zero-Context Developer Onboarding** Maps high-level natural-language questions to exact implementation functions and structural call graphs.
- **Behavior-Guided Bug & Regression Triage** Worked examples in issue tickets verify whether candidate functions produce expected outputs before code review.
- **Real-Time CI/CD Incremental Re-Indexing (P1)** Re-embeds only changed AST chunks on pull requests, updating semantic search in ~1.5 s without blocking workflows.
- **Contamination-Free Code Archaeology (Bonus)** Retrieves historical functions from specific git commits without cross-version context leakage or silent fallback to HEAD.
- **Trustworthy Developer Tooling** Returns verifiable source code spans with exact file/line evidence rather than uncheckable generative model text.

Innovation highlights, results and limitations

- **P0 Retrieval Accuracy (CoIR AppsRetrieval Benchmark)** NDCG@10: 0.96558 · MRR@10: 0.95857 · Recall@10: 0.98619. Dense baseline: 0.93756; behavioral execution delivers +0.02802 lift (removes 44.9% of residual error).
- **P1 Incremental Indexing (Kaggle 4-vCPU Intel Xeon)** Full rebuild: 103.4 s (50 chunks) vs. Unchanged update: 1.51 s (50 reused) → ~68x speedup (100% equivalence, min cosine 1.0).
- **Bonus Cross-Version Selection (12 Lineages across 3 Versions)** 100% correct variant selection (1.0) with execution verification vs. 8.33% (0.0833) for raw semantic retrieval (~12x accuracy gain).
- **Honest Limitations (Academic Integrity)** Passing visible test cases does not prove formal correctness; subprocess limits, not OS hypervisor; APPS pre-training exposure transparently documented.

What’s next

- **Unseen Python Evaluation Readiness** Generalize execution verification to arbitrary test runners (pytest, unittest) for the judges' unreleased dataset.
- **Sub-200ms CPU Retrieval (INT8 / ONNX)** Implement static integer quantization and ONNX Runtime CPU execution to eliminate dense latency with zero GPU dependence.
- **MicroVM Execution Sandboxing** Migrate from OS-level subprocess limits to Firecracker / gVisor microVMs for secure, multi-tenant repository code execution.
- **Cross-File Structural Call Graphs** Deepen static analysis with whole-repository points-to analysis and cross-module import tracking across monorepos.

Brownie points slide(differentiation)

- **Agentic Multi-Signal Routing** Cheap lexical and symbol passes resolve simple queries in 0.4 ms; expensive dense inference is invoked only when semantic reasoning is required.
- **Behavioral Post-Processing Verification** Validates candidate code semantics via executable test cases, solving near-miss ranking where text embeddings fail.
- **Content-Addressed Vector Cache** Vectors are keyed on chunk hash and encoder recipe—immune to file renames, line shifts, and directory reorganization.
- **Strict Version Provenance & Lineage** Groups evolving snippets using 5-shingles and Jaccard clustering, ranking working variants first across Git history.
- **Academic Research Integrity** Preserved negative results (learned reranker failed bootstrap gate, so it was rejected); enforced cache=None in official MTEB runs.

Checklist - Submission materials

- **[x] Working Prototype Source Code** Full source code merged on main (commit 2024ca1) with reproducible setup instructions and 571 passing tests.
- **[x] Requirements Files** Both requirement.txt (singular, checklist compliant) and requirements-submission.txt pinned and verified.
- **[x] Presentation Deck & PDF** Completed 12-slide presentation (VITV_Doctors_Submission.pptx) and matching PDF (VITV_Doctors_Submission.pdf).
- **[x] AI Disclosure & Official Release Tag** Official LangAI3.0_AI_Disclosure.docx completed; released under tag PRISM_GENAI_HACKATHON_Y2026 with MTEB results.
- **[x] Product Demo Video** 5-minute product walkthrough video recorded and linked in README.md and release assets.

Thank you

CodeDoctor / VITV / Doctors

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India